

# Weekly Learning Debrief

MIS410 — Data Analytics Certificate | Week of July 26, 2026

Topics: MySQL + Python connection, Jupyter Notebook workflow, pandas data cleaning, data organization, matplotlib visualization

---

## Key Concepts Learned

### Variables & Naming

- A variable is a labeled box — Python doesn't care what it's named, as long as the same name is used consistently everywhere afterward.
- Naming conventions like `_df` are hints for humans, not rules for Python — they signal “this holds a DataFrame” vs. something narrower like a Series.
- Renaming something partway through code means every reference to it must match, or Python throws a `NameError`.

### The Period ( `.` ) vs. Underscore ( `_` ) Distinction

- **Underscore** connects words within a name — `example_df`, `city_sales`.
- **Period** connects a thing to an action being done to it — `.head()`, `.median()`, `.fillna()`, `.sort_values()`.
- Rule of thumb: naming = underscore, doing = period.

### The Equals Sign ( `=` )

- Always means: take what's on the right, save it into what's on the left.
- Without it, a calculation displays once and disappears — it doesn't permanently update the data.

## Pandas Toolkit Built This Week

| Method                                          | What it does                                                            |
|-------------------------------------------------|-------------------------------------------------------------------------|
| <code>.isnull().sum()</code>                    | Counts missing values per column                                        |
| <code>.fillna(value)</code>                     | Fills blanks with a specific value (median, "Unknown", etc.)            |
| <code>.duplicated()</code>                      | Flags exact-match duplicate rows                                        |
| <code>.drop_duplicates()</code>                 | Removes those duplicates                                                |
| <code>.str.title()</code>                       | Standardizes text capitalization                                        |
| <code>.unique()</code>                          | Lists distinct values in a column (like Excel's UNIQUE)                 |
| <code>.groupby("col")<br/>["col2"].sum()</code> | Groups rows and totals a column per group (like a pivot table)          |
| <code>.sort_values(ascending=False)</code>      | Sorts largest-to-smallest (pandas has no <code>descending=True</code> ) |
| <code>.plot(kind="bar")</code>                  | Turns data into a chart                                                 |

## Full Pipeline Built

MySQL database created → CSV generated → SQLAlchemy bridged pandas to MySQL → data loaded into a table → SQL query written → query run through Python into a DataFrame → data cleaned → data organized/grouped → visualized → cleaned data exported → all documented in Markdown.

This is a real end-to-end data pipeline: source → database → query → clean → analyze → visualize.

## Debugging Patterns Worth Remembering

- **“Not recognized” errors** — often a PATH issue; `python -m [package]` sidesteps it.
- **Cell execution numbers** show actual run order, not page position — mismatched/out-of-sequence numbers are the first clue when output looks wrong.
- **Code cell vs. Markdown cell** — Markdown is notes (plain English, no execution), Code is instructions (must be valid Python). Mixing them up throws the same red syntax error.
- **Special characters in passwords** (like `@`) can break connection strings and need URL-encoding (`%40`).
- **Dependent columns** (like `TotalPrice` depending on `PricePerUnit`) don't auto-update — recalculate manually after fixing the source column.

## Biggest “Aha” Moments

- Markdown cells = notes, not syntax to execute.
- `example_df = ...` permanently saves changes — it's not previewing, it's overwriting.
- Mean vs. median matters because of outliers; median is safer for messy real-world data.
- `.str` only works on actual text data, not real numbers — even if a number looks like text.
- Python and Jupyter often infer more than SQL or Excel, which tend to demand fully explicit instructions.

---

*Next steps: bring this same format back each week to track growth toward the Mod 8 goal of working independently.*